

Greedy
Answer Template:
Proof: exchange argument
1. Define.
2. Greedy Solution G={g₁,g₂,...,g_k}, ordered by the greedy sorting criterion
3. An Optimal Solution OPT={o₁,o₂,...,o_m}, sorted by the **same** criterion
4. We want to show **k=m**.
5. Compare solutions.
6. Find the largest index r such that g₁=o₁, g₂=o₂, ..., g_r=o_r (i.e. the first r elements in OPT and G are the same but not the (r+1)-th element)
7. Case 1: r=min(k,m),
8. If k=m, we are done
9. If k>m → k<m and k>m both yield contradiction.
10. Case 2: r<min(k,m), the first mismatch is at index r+1, g_{r+1}≠o_{r+1}.
11. Exchange Pieces.
12. Argue that the greedy rule forces (measure)(g_{r+1}) ≤ (measure)(o_{r+1}), (i.e. Finish time)
13. Replacing o_{r+1} by g_{r+1} in OPT, OPT'={o₁, o₂, ..., o_r, g_{r+1}, o_{r+2}, ..., o_m}.
14. Prove cost doesn't increase (g_{r+1} does not overlap any of {o₁,...,o_r} and {o_{r+2}, ...,o_m}).
15. OPT' still feasible and optimal. |OPT'|=|OPT|=m.
16. Iterate Until Full Alignment.
17. Replace OPT with OPT'. Now the first r+1 elements in OPT and G are the same.
18. If r+1=min(k,m) → Case 1
19. Otherwise, we have a new mismatch at index (r+1)+1. Continue iteratively until the first min(k,m) elements match. In every swap, the solution stays feasible and keeps size m.
20. Conclusion:
21. every step maintains feasibility and preserves the number of intervals,
22. the "r=min(k,m)" case rules out k<m and k>m, we deduce k=m.
23. The greedy solution G has the same number of elements as any OPT, so G is optimal.

Applications List (definition, solution, time):
• Interval Scheduling: O(nlogn)
• Given a set of jobs, each with a start and finish time, select a **maximum-size subset of non-overlapping jobs**. → Sort jobs by earliest finish time, then scan through, picking each job that starts after the last one you selected.
• Knapsack(0-1):
• Given n items each with weight w_i and value b_i and capacity W, choose a subset to **maximize total value under weight ≤ W**, but **take each item whole or not at all**; pseudo-polynomial (polynomial in size of numbers not their bit length) DP solution in O(nW). polynomial in n and the numeric target W, but not polynomial in logW.
• Optimization ver.: NP-hard, Decision ver.: NP-Complete
• Fractional Knapsack: O(nlogn).
• allows taking partial items (difference), "picking item with highest b_i/w_i" rule is provably optimal
• Interval Partitioning: O(nlogn).
• Given n intervals, assign them to the **minimum number of "rooms"** so that no two overlapping intervals share a room → Sort intervals by start time, use a min-heap of end times, if the earliest-ending room is free (end ≤ start), reuse it; otherwise open a new room.
• Scheduling to Minimize Lateness: O(nlogn)
• Given n jobs each with processing time t_j and due date d_j, find an order that **minimizes the maximum lateness L = max_j[(f_j - d_j)]** → Sort jobs in nondecreasing order of due date.
• Shortest path: Dijkstra, Binary-heap → O((n + m) log n); Fibonacci-heap → O(m+nlogn)
• Minimum spanning trees: Given a connected weighted graph, find a subset of edges of minimum total weight that connects all vertices. → Kruskal: sort edges and union-find, O(m log m)≈O(m log n). Prim's: priority-queue over vertices, O(m + n log n)

Coping with hardness
(sacrifice 1 of: optimality,poly-time,arbitrary instances)
Solving NP-Hard Problems on restricted input instances:
Independent Set on Trees: Greedy, O(n) in postorder.
Weighted IS on Trees: DP, O(n) in postorder.
Solving restricted instances
Finding Small Vertex Covers: Let (u,v) be an edge of G. G has a vertex cover of size≤k iff at least one of G\{u} and G\{v} has a vertex cover of size≤k-1. Can be solved in O(2^kkn) time.
Approximating the optimal solution

- Sacrifice optimality, but still run in polynomial time.
- Approx ratio=Cost of our solution/Cost of optimal solution
- An approximation algorithm for a minimization problem comes with an approximation guarantee:
 - Approximation ratio ≤ c
 - Cost of our solution ≤ c · cost of optimal solution
- **Vertex Cover:** S is a **2-approximation**.
- **Weighted VC:** For any vertex cover S and any fair prices
- **Pricing method.** Set prices and find vertex cover simultaneously. a **2-approximation**.
- **Load Balancing:** makespan = $L = \max L_i$
- 1. **List Scheduling:** fixed order of processing time, assign each job to machine with smallest load so far. O(n log n) using a priority queue.
- 2. **Longest processing time (LPT):** Sort n jobs in descending order of processing time, and then run list scheduling algorithm. a **4/3-approximation**.

Greedy
Answer Template: General Idea
0. Preprocessing: After pre-processing, we will use the recursive function X which takes input A and outputs R in A.
1. Divide

- a. **Base cases:** trivial, can be solved in constant runtime, always reachable. n = 1 when there is only one element.
- b. **Substances:** smaller, independent, the exact same problem. Divide A into left and right half as AL and AR.

2. Delegate: Recursively solve substances independently. Apply X on AL and AR and let L and R be the majority element returned by Major(AL) and Major(AR).
3. Combine: Spot a general pattern from base case, apply to combining general substance solutions. Recursive characterization of X contains 3 cases. If... return... Else, extra computation, return...

Proof: Induction
1. Base case: n=1
2. Induction hypothesis: Assume that the algorithm is correct for all k, with 1 ≤ k ≤ n - 1.
3. Inductive step: Prove the algorithm is correct for k=n.
Time: The number of recursive calls (k). The size of the input passed to each recursive call relative to the current size of the input(n/2). The amount of work performed at each recursive call relative to the size of the input (f(n))
$$T(n) = \begin{cases} a & \text{if } n \leq d \\ c & \text{if } n > d \end{cases}$$

- 1. if $f(n) = O(n^{\log_a(a-\epsilon)})$ for $\epsilon > 0$ then $T(n) = O(n^{\log_a(a-\epsilon)})$,
- 2. if $f(n) = O(n^{\log_a(a)} \log^k n)$ for $k > 0$ then $T(n) = O(n^{\log_a(a)} \log^{k+1} n)$,
- 3. if $f(n) = O(n^{\log_a(a+\epsilon)})$ and $a f(n/b) \leq \delta f(n)$ for $\epsilon > 0$ and $\delta < 1$ then $T(n) = O(f(n))$.

Unrolling: $T(n) = 2^k T(n/2^k) + O(n) = c \cdot n + 2^k c \cdot (n/2^k) + 4^k c \cdot (n/4^k) + \dots$

- $T(n/(2^k k))$ reaches the base case when $n/(2^k k) = 1$,
- $\Rightarrow k = \log_2(n)$. Substituting $k = \log_2(n)$.
- $T(n) = n^* T(1) + c \cdot n [1/2 + 1/4 + \dots + 1/n]$

Useful Formula: $T(n) = 2T(n/2) + \log n = O(n \log n)$ (re-sort the combined n elements)
 $T(n) = 2 T(n/2) + O(n \log n) \quad T(n) = O(n \log n)$ (Merge sort)
 $T(n) = 2 T(n/2) + O(\log n) \quad T(n) = O(n)$
 $T(n) = 2 T(n/2) + O(1) \quad T(n) = O(n)$
 $T(n) = T(n/2) + O(n) \quad T(n) = O(n)$
 $T(n) = T(n/2) + O(1) \quad T(n) = O(\log n)$ (Binary Search)
 $T(n) = T(n/1) + O(n) \quad T(n) = O(n^2)$
 $T(n) = T(n-1) + O(1) \quad T(n) = O(n)$

Applications List (general idea, time):
• Maximum Contiguous Subarray: $T(n) = 2 T(n/2) + O(n) \rightarrow T(n) = O(n \log n)$.
Given an array A[] of n numbers, find the **maximum sum found in any contiguous subarray**.
1. MCS in A[1..n/2] & A[n/2+1..n]: Recursively compute the maximum contiguous-subarray sum in the left & right half.
2. MCS that spans across A[n/2]
a. Scan left from index n/2 to find the largest suffix-sum ending at A[n/2].
b. Scan right from index n/2+1 to find the largest prefix-sum starting at A[n/2+1].
c. Add those two values to get the best "cross-middle" sum.
• Counting Inversions $T(n) = 2 T(n/2) + O(n)$ (O(nlogn)).
1. CI in A[1..n/2] & A[n/2+1..n]: Recursively count inversions in the left & right half and produce a sorted left & right subarray.
2. CI that spans across A[n/2]: Merge the two already-sorted halves into one sorted array, and during the merge:
a. Whenever you take an element from the right half because it is smaller than the current left-half element, add the number of remaining items in the left half to your cross-inversion count.
b. The merged output is a fully sorted A[1..n], and the total cross-inversions is exactly the number of left-vs-right inversions.
• Closest-Pair: Preprocessing: O(n log n). $T(n) = 2T(n/2) + O(n) = O(n \log n)$
• Given n points in the plane, find a pair with smallest Euclidean distance between them. Only need to consider points within 5 of line L. Sort points in 2d-strip by their y coordinate. Only check distances of those within 7 positions in sorted list.
• Majority Element: $T(n) = 2T(n/2) + O(n) = O(n \log n)$
• If the array has a single element, we return that element as the majority element. Otherwise, we break the input array A into two halves, recursively call the algorithm on each half, and then test in A if either of the elements returned by the recursive calls is indeed a majority element of A. If that is the case we return the majority element, otherwise, we report that there is "no majority element".
• Correctness: If there is no majority element of A then the algorithm must return "no majority element" since no matter what the recursive call returns, we always check if an element is indeed a majority element. Otherwise, if there is a majority element x in A we are guaranteed that one of our recursive calls will identify x (by part a) and the subsequent check will lead the algorithm to return x.
• Largest Complete Subtree: $LCS(\text{root}) = \min(LCS(L), LCS(R)) + 1$.

Base Exchange Rule: $\log_a(x) = (\log_b(x))/(\log_b(a))$
Product Rule: $\log_a(xy) = \log_a(x) + \log_a(y)$
Power Rule: $\log_a(x^b) = b \log_a(x)$
Graph
1. Complete Graphs: is one where every two vertices are connected with an edge.
2. DFS Tree: a simple path; BFS Layers: L₀ = {s}, L_i = V\{s}
3. Traversal Time Complexities (Adjacency-List): BFS & DFS: O(n + m)
4. Bridge (cut-edge): an edge whose removal increases the number of connected components.
5. Trees: connected, acyclic undirected graphs.
6. Every edge in a tree is a bridge. A tree on n vertices has exactly n - 1 edges.
7. For every simple undirected graph, $\sum \deg(v) = 2|E|$.

constant k
Weight wgt Value val
wgt[i-1] val[i-1]
1 5
2 11
3 15
wgt[i-1] val[i-1]
1 5
2 11
3 15

Greedy
Answer Template:
General Idea
1. Define Substances OPT[i,j] =
2. Good substances should satisfy: *overlapping
3. There is only a polynomial number of them
4. The original instance is either a substance in itself or can easily be derived from one
5. There exists a natural ordering between the substances (i.e., your dependency graph should be directed and acyclic)
6. You should be able to compute the solution to a substance from some number of smaller substances using a well-defined recurrence
7. Define Recurrence OPT[i,j] = max{...}
8. Assume you include something in your optimal solution
9. What does that say about the structure of the remainder of the solution?
10. Formulate this in terms of substances.
11. Define Base case (satisfies the 3 following properties):
12. Trivial to solve without the need for smaller substances.
13. Can be solved in constant time
14. In a top-down approach, ensure it will always be reached; In a bottom-up approach, ensure you will never require a smaller substance to build up your solution.
15. Evaluate substances in the right order
16. Ensure that when computing the solution to a substance, the substances it depends on have previously been solved.
Proof: Induction (0-1 knapsack)

1. base cases are sufficient. Show every path of dependencies hits a base case you've defined. The base cases are sufficient as each OPT[i, w] depends on some OPT(i - 1, w). Thus, the recurrence will always end up at OPT(0, w) for some w.
2. base cases are correct. Show each base case's value really satisfies the problem statement. We have OPT(0, w) = 0 since the most value that can be achieved with no items is 0.
3. recurrence for OPT(i) is correct. (Inductive Hypothesis + step) Let S(i, w) be the optimal solution for the first i items with weight limit w. The recurrence for OPT(i, w) is correct because:
• If w_i > w, then S(i, w) cannot contain item i and so is a subset of the first i - 1 items with weight at most w, so S(i, w) = S(i - 1, w) and OPT(i, w) = OPT(i - 1, w)
• If w_i ≤ w, then either item i is in S(i, w) or not:
• if item i is in S(i, w), then the remaining items of S(i, w) is a subset of the first i - 1 items with weight at most w - w_i, so S(i, w) = {i} ∪ S(i - 1, w - w_i) and so OPT(i, w) = v_i + OPT(i - 1, w - w_i).
• else, S(i, w) is a subset of the first i - 1 items with weight at most w, so S(i, w) = S(i - 1, w) and OPT(i, w) = OPT(i - 1, w).
• Finally, the optimal value for the original problem is OPT(n, W) as by definition of OPT, this is the most value that can be achieved by a subset of all n items with a weight limit of W.
4. Evaluation Order is correct. Show that your loops or recursion really do compute the smaller subproblems before the larger ones. We fill the DP table in row-major order. Our recurrence only ever reads entries from row i-1. Since when the outer loop is at i, we have already completed row i-1, by the time we compute each OPT(i,w), all needed OPT(i-1,*) are available. Hence the evaluation order respects the dependency graph.

Time & Space 1. The number of substances you need to compute to calculate the solution to your original instance. 2. The time it takes to compute each substance.
• Space: 1. The space required is dominated by the number of subproblems × space per subproblem. We define OPT[i,w] (0 ≤ i ≤ n, 0 ≤ w ≤ W) and store these in an (n+1) × (W+1) array. 2. Total space: O((n+1)(W+1)) = O(nW).
• Time: 1. Setting every entry in the (n+1) × (W+1) table to 0 takes O((n+1)(W+1)) = O(nW). 2. Fill DP table. For each item i=1..n and each capacity w=0..W, compute OPT[i,w]=max{OPT[i-1,w], v_i+OPT[i-1, w-w_i]} in O(1) time (a couple of array lookups, one addition, one comparison). Total for this phase: O(nW). 3. Reading off OPT(n,W) (the final answer) takes O(1). 4. Thus the total time is O(nW)+O(nW)+O(1) = O(nW).
tree-DP e.g. Maximum-weight independent set on a tree
1. Root the tree. Pick any node as root (call it r), and direct all edges "away" from r. Now every node u has a (possibly empty) set of children C(u).
2. Decide your states. Think about what choices you have at a node u, and what information you need to pass up to its parent.
3. Write the recurrence. Express each DP value at u in terms of the DP values of its children {v ∈ C(u)}. That usually means summing over children, taking maxima or minima, maybe adding edge weights, etc.
4. Base: u is a leaf. dp[u][0]=0, dp[u][1]=w_u.
5. Recurrence: Suppose C(u)={v₁,v₂,...}.
6. dp[u][0] = best total weight in u's subtree if we do not pick u.
7. dp[u][1] = best total weight in u's subtree if we do pick u.
8. dp[u][0] = ∑_{v ∈ C(u)} max(dp[v][0], dp[v][1])
9. dp[u][1] = w_u + ∑_{v ∈ C(u)} dp[v][0].
10. Root: max(dp[r][0], dp[r][1]).
11. Traverse in post-order. Do a DFS from the root. When you return from all of u's children, you compute u's DP values in O(deg(u)). No cycles means you never revisit a node. ∑deg(u)=2|E|=O(n).

Weight wgt Value val
wgt[i-1] val[i-1]
1 5
2 11
3 15
wgt[i-1] val[i-1]
1 5
2 11
3 15

Applications List (sub-instances, recurrence, base cases, time):
1D dp
• Maximum-Sum Contiguous Subarray: O(n)
• Include or exclude A[i]? → pick max of two
• OPT[i]=value of the optimal contiguous subarray ending at i (possibly empty)
• OPT(i) = max_{1 ≤ j ≤ i} {A[j] + ... + A[i]}, 0.
• OPT(i) = { max{A[i], 0}, i = 1, max{OPT(i-1) + A[i], 0}, i > 1.
• Weighted Interval Scheduling: O(nlogn); if pre-sorted: O(n)
• Include or exclude job j? → two-way choice
• OPT(i)=max-weight of a compatible subset chosen from jobs 1, 2, ..., i.
• OPT(j) = max-weight of a compatible subset chosen from jobs 1, 2, ..., j
• OPT(j) = { 0, j = 0, max{w_j + OPT(p(j)), OPT(j-1)}, j > 0.
• Longest Increasing Subsequence: O(n²)
• Split at some j < i that comes before i
• OPT(i)=length of the longest increasing subsequence ending at i.
• OPT(i) = 1 + max_{1 ≤ j < i} {OPT(j)},
OPT(1) = 1 (every single element by itself).
• Coin Change 1(Sample): O(nk), k=number of coin categories
• derive it from smaller amounts by "try every coin"
• OPT(i)=minimum number of coins needed to make amount i
• OPT(i) = { 0, n = 0, -1, n < 0, min_{coin ∈ coins} {dp(n - coin) + 1}, n > 0.

2D dp
• 0-1 Knapsack: O(nW)
• Include or exclude item i (change w or not)? → two-way choice
• iterate w downward from W to w_i
• OPT(i,w)=max value of a subset of items {1,2,...,i} with weight limit w for 0 ≤ i ≤ n and 0 ≤ w ≤ W.
• OPT[i,w] = { 0 if i=0, OPT[i-1,w] if w_i > w, max{OPT[i-1,w], b_i+OPT[i-1,w-w_i]} otherwise
• Unbounded Knapsack O(nW)
• Include or exclude item i (change w or not)? → two-way choice
• iterate w upward from w_i to W
• OPT(i,w)=max value using item types 1,2,...,i with weight limit w.
• OPT(i,w) = { 0, i = 0, OPT(i-1, w), w < w<sub>i}, max{OPT(i-1, w), v_i + OPT(i, w - w_i)}, otherwise.
• Longest Common Subsequence: O(nm)
• Compare X[i] vs Y[j] → move NW(↖) (match) or N/W
• OPT(i,j)=length of LCS(X[1..i], Y[1..j])
• Find LCS of any prefixes of X and Y (not just of same length)
• OPT(i,j) = { 0 if i = 0 or j = 0, max{OPT(i-1,j-1) + 1, OPT(i-1,j), OPT(i,j-1)} if X[i] = Y[j], max{OPT(i-1,j-1), OPT(i-1,j)} if X[i] ≠ Y[j]</sub>

• Shortest paths in graphs with negative edge weights
• Working space: O(nl) Total space (including input): O(m+n)
• Running time: O(mn) worst case
• Include/Exclude last edge
• OPT(i,v)=length of shortest v-t path P using at most i edges.
• OPT(i,v) = { 0 if i=0 and v=t, ∞ if i=0 and v≠t, min{OPT[i-1,v], min_{{v,w} ∈ E} {OPT[i-1,w]+c_{v,w}}} otherwise
• Coin Change 2: In how many ways can you combine coins to sum to N? (similar to unbounded knapsack) O(nk), k=number of coin categories
• Include or exclude coin i (change w or not)? → two-way choice
• OPT[i][v]=number of ways using the first i coins (coins[1..i]) to make amount v.
• OPT[i][v] = { 1, v = 0 and i = 0, 0, v ≤ 0 and i ≠ 0, OPT[i-1][v] + OPT[i][v - w_{i}], otherwise.}

dp over intervals
• RNA secondary structure: Time: O(n³) Space: O(n²)
• Split interval at some k paired with j
• OPT(i,j)=max number of base-pairs in a secondary structure of the substring [i..j]
• OPT(i,j) = { 0, i ≥ j - 4, max_{1 ≤ t < j - 4} {1 + OPT(i, t - 1) + OPT(t + 1, j - 1)} i < j - 4.
• Segmented Least Squares: O(n²)
• Choose where to start the last segment
• OPT(i)=minimum cost for points p₁, p₂, ..., p_j
• e(i, j)=minimum sum of squares for points p₁, p₁+1, ..., p_j
• C = fixed penalty cost per segment.
• OPT(j) = { 0, j = 0, min_{1 ≤ i < j} {e(i, j) + c + OPT(i-1)}, j > 0.

1. **Flow network on a directed graph:**

- A flow network on a directed graph has:
 - Two distinct nodes the source, s , and the sink, t .
 - Each edge has a capacity $c(e)$, representing how much flow it can carry.
 - Flow $f(e)$ through each edge. An edge is saturated if $f(e)=c(e)$.

A s - t flow also must satisfy two constraints:

- Capacity:** For each $e \in E$, $0 \leq f(e) \leq c(e)$
- Conservation:** For each $v \in V - \{s, t\}$, $\sum_{e \in \text{in}(v)} f(e) = \sum_{e \in \text{out}(v)} f(e)$

The value of a flow f is:

$$v(f) = \sum_{e \in \text{in}(t)} f(e) = \sum_{e \in \text{out}(s)} f(e)$$

A s - t cut

- is a partition (A, B) of V with $s \in A$ and $t \in B$.
- Capacity: $\text{cap}(A, B) = \sum_{e \in \text{cut}(A, B)} c(e)$

2. **Flow-Cut Bounds & Duality**

Max flow problem: in both P and NP

Weak Duality (Max-Flow/Min-cut): $v(f) \leq \text{cap}(A, B)$, $v(f) \leq \text{capacity}(A, B)$.

- Corollary: If $v(f) = \text{cap}(A, B)$, f is a max flow, (A, B) is a min cut.

Flow Value Lemma: $v(f) = f_{\text{in}}(\text{out}(A)) - f_{\text{in}}(\text{in}(A))$

Max-Flow Min-Cut theorem: max $v(f)$ = min $\text{cap}(A, B)$.

Bounds for Any Cut: $v(f) \leq \text{cap}(A, B)$ (so "max flow value $\leq$ capacity of any cut").

3. **Residual Network & Augmenting Paths**

Residual Graph G_f :

- If $c(e) - f(e) > 0$ $\Rightarrow$ Forward edge (u, v) with capacity $c(e) - f(e)$
- If $f(e) > 0$ $\Rightarrow$ Backward edge (v, u) with capacity $f(e)$

Augmenting Path: Any path from s to t in G_f

- f is maximum $\Leftrightarrow$ there is no augmenting path in G_f
- If there is an augmenting path P , then we can push extra flow along that path.
- b -bottleneck $P(f)$: min capacity on edges of P in residual network.
- Forward edges: increase $f(e)$ by b . Backward edges: decrease $f(e)$ by b .

4. **Ford-Fulkerson & Variants** (# of iterations, Cost per iteration, Time)

FF: $\leq F$ (value of max flow); $O(m + n)$; $O((m + n)F) = O(mF)$

- Choose any augmenting path using DFS;

Edmonds-Karp #1: $O(m \log F)$; $O(m \log n)$ (greedy); $O(m^2 \log n \cdot \log F)$

- Choose maximum bottleneck augmenting path using priority-queue/Dijkstra

Capacity-Scaled: $O(m \log F)$; $O(m)$; $O(m^2 \log F)$

- Choose $1/2$ -approximate max bottleneck path via restricting to edges of capacity $\geq \Delta$ using DFS (Graph very dense) (Maximize throughput)

Edmonds-Karp #2: $O(n^3 m)$; $O(n^3 m)$; $O(n^3 m \cdot (m+n) \log F) = O(n^4 m^2)$

- Choose shortest augmenting path using BFS (Need strict poly-time)

5. **Verification & Complexity**

Verify Max Flow in $O(V+E)$

- Build residual graph G_f . 2. Run one BFS/DFS from s .
- If you reach t : there is an augmenting path $\Rightarrow$ not max. Otherwise $\Rightarrow$ is max.

Integrality Theorem: If all capacities are integers, there exists a max flow f with all $f(e) \in \mathbb{Z}$.

6. **Circulations & Extensions**

A circulation is a flow network but with supplies and demands. Each vertex v has a demand $d(v)$. demand if $d(v) > 0$, supply if $d(v) < 0$, transshipment if $d(v) = 0$.

A valid circulation also must satisfy two constraints:

- Capacity: For each $e \in E$, $0 \leq f(e) \leq c(e)$
- Conservation: For each $v \in V - \{s, t\}$, $\sum_{e \in \text{in}(v)} f(e) - \sum_{e \in \text{out}(v)} f(e) = d(v)$

Necessary condition: sum of supplies = sum of demands.

Reduce to max flow:

$$-\sum_{v: d(v) > 0} d(v) \geq -\sum_{v: d(v) < 0} d(v) \Rightarrow \sum_{v: d(v) < 0} d(v) = 0$$

- "hook up" all the supply vertices to a super- s , demand vertices to a super- t
- For each v with $d(v) < 0$, add edge (s, v) with capacity $-d(v)$; same for t .
- Use flow networks to model: **G has circulation iff G' has max flow value D.**

Circulation with supply/demand and lower bounds

- Capacity: For each $e \in E$, $l(e) \leq f(e) \leq c(e)$

7. **Self-Check Questions**

- Matching vs Vertex-Cover (Undirected): $| \text{max matching} | \leq | \text{min vertex cover} |$.
- A max flow f and a min cut (A, B) with $v(f) > 0$: $f_{\text{in}}(\text{in}(A)) = 0$, $f_{\text{out}}(\text{out}(A)) = 0$

Applications List (definition, maximum flow formulation, equivalence, time):

Bipartite matching:

- Input: undirected, bipartite graph $G=(L, R, E)$
- Max matching: MCE is a matching if each node appears in at most one edge in M (matching constraint)
- Equivalence: Max cardinality matching in $G =$ value of max flow in G' .

$\Rightarrow$ If there is a matching of size k , then there is a flow of value at least k .

- Matching constraints ensure that we satisfy flow constraints
- Size of matching ensure that flow value equals size of matching

$\Leftarrow$ If there is a flow of value k , then there is a matching of size at least k .

- Flow integrality ensures we do not choose edges fractionally
- Constraints of flow network (capacity, conservation) ensures we satisfy matching constraints (each vertex incident to at most one matching edge)
- Flow Value Lemma shows that matching has same size as value of flow

Running Time: FF: $O(mF) = O(mn)$; EK: $O(m^2 \log F) = O(m^2 \log n)$.

Perfect Matching: A matching MCE is perfect if each node appears in exactly one edge in M .

Marriage Theorem: Let $G=(L, R, E)$ be a bipartite graph with $|L|=|R|$. Then, G has a perfect matching iff $|N(S)| \geq |S|$ for all subsets $S \subseteq L$.

Edge disjoint paths - Disjoint path problem: Given a digraph $G=(V, E)$ and two nodes s and t , find the max number of edge-disjoint s - t paths.

- Definition: Two paths are edge-disjoint if they have no edge in common.
- Equivalence: Max number of edge-disjoint s - t paths = max flow value.

Network Connectivity (Min-Cut):

- Definition: Given a digraph $G=(V, E)$ and two nodes s and t , find min number of edges whose removal disconnects t from s .
- Theorem: The max number of edge-disjoint s - t paths is equal to the min number of edges whose removal disconnects t from s .

Baseball Elimination:

- Flow from game node (x, y) to team node (x) equals # of matches between teams x and y that team x wins.
- Flow from a team node (x) to the sink ensures no other team wins too many games. ($w_3 + r_3 - w_4$ is exactly "Team 3's best-possible final wins" ($w_3 + r_3$) minus "Team 4's current wins" (w_4))
- Team i is not eliminated iff there is a flow in G of value at least the total number of remaining games between all pairs of teams.

Image Segmentation; Survey Design

2. **Properties Translation using components (reduction/network construction)**

Construct a flow network $G=(V, E)$ with capacity function c as follows:

- Add a source vertex s and a sink vertex t to V .
- For each computer peP , add a vertex u_p to V and an edge (s, u_p) to E , and set $c(s, u_p) = 1$.
- For each base station beB , add a vertex v_b to V and an edge (v_b, t) to E , and set $c(v_b, t) = 1$.
- For each computer-station pair $(p, b) \in P \times B$, if $|x_p - x_b| \leq r_p$, then add an edge (u_p, v_b) to E , and set $c(u_p, v_b) = \infty$.

Finally, set the target flow value to $K = |P|$. If its value equals, then all computers can be assigned (output YES); otherwise output NO.

3. **State which algorithm you use to compute the max flow.**

Use Ford-Fulkerson to compute the maximum flow K in G .

- If the resulting max-flow K is not equal to $|P|$, then no feasible assignment exists.

4. **Solution Translation**

Feasibility: If max-flow = $|P|$, every u_p sends exactly 1 unit to some v_b .

Reconstruction: For each computer p , find the station b such that the edge $u_p \rightarrow v_b$ carries 1 unit of flow. Assign p to b .

Proof

$\Rightarrow$ If there exists a feasible assignment in our original problem instance then there must exist a max flow of value n in our reduced max flow instance (e.g. a base station that is within range for every computer).

1. **Assume assignment and define a flow instance.**

- Assume there is a valid assignment $g: P \rightarrow B$ with each station's load $L_b = \begin{cases} 1, & g(p) = b, \\ 0, & \text{otherwise.} \end{cases}$
- Define a flow f on the max flow instance as follows:
 - For each computer $p \in P$, set $c(s, u_p) = 1$.
 - For each computer-station pair $(p, b) \in P \times B$ with an edge (u_p, v_b) , set $f(u_p, v_b) = 1$.
 - For each base station $b \in B$, set $f(v_b, t) = L_b$.
- By inspection of the reduction, the above steps assign a flow value to each edge in G .
- So f saturates the source cut, so $v(f) = c(s, V(s)) = |P| = K$. $v(f) = \sum_{p \in P} f(s, u_p) = |P| = K$.

2. **Check feasibility (a. capacity; b. integrality; c. conservation)**

To establish that f is a valid flow, we need to check the flow constraints.

- $f(e) \leq c(e)$ for every edge $e \in E$:
 - (s, u_p) are all assigned flow equal to their capacity.
 - (u_p, v_b) all have capacity ∞ .
 - (v_b, t) : capacity L_b is constituted of 1 from (u_p, v_b) s.t. $g(p)=b$: The amount of p , by assumption, can't be greater than L_b .
- $f_{\text{in}}(u_p) = f_{\text{out}}(u_p)$ for every $u_p \in V - \{s, t\}$:
 - At u_p : $f_{\text{in}}(u_p) = 1$; $(u_p, v_{g(p)})$ is guaranteed to exist in G ; $f(u_p, v_b) = 1 \Rightarrow f_{\text{out}}(u_p) = 1$
 - At v_b : in-flow from computer equals out-flow to t by definition.
- Integrality (and non-negativity): all $f(e)$ are integers ≥ 0 .

3. **Conclusion**

- Hence all flow-constraints are met, so f is a valid integral flow of value K .

$\Leftarrow$ If there exists a max flow of value n in our reduced max flow instance then there must exist a feasible assignment in our original problem (e.g. it must satisfy all edge capacity and flow conservation requirements)

1. **Assume the flow (Max-flow existence).**

- Assume the flow network admits an integral flow f of value $K = |P|$.
- By construction, K equals the capacity of the cut $c(s), V(s)$, so the max flow is $|P|$.

2. **Decoding to a solution**

- Flow integrality.** K is the capacity of the source cut, and by flow integrality theorem, we can assume that all flow values assigned to edges by f are integers.
- One unit per computer.** Look at a computer-node u_p . The only incoming edge to u_p is (s, u_p) of capacity 1, and the only outgoing edges are (u_p, v_b) .
 - Since the cut $c(s), V(s)$ is saturated, every (s, u_p) must carry exactly 1 unit.
 - From the flow conservation at u_p , e can deduce that $f_{\text{out}}(u_p) = f_{\text{in}}(u_p) = 1$ for each peP .
- c. Define the computer-station assignment $g: P \rightarrow B$**
 - $f_{\text{out}}(u_p) = 1 \Rightarrow u_p$ sends 1 unit in total
 - Flow integrality $\Rightarrow$ each outgoing (u_p, v_b) has integral flow.
 - $\Rightarrow$ exactly one edge (u_p, v_b) has a flow value of 1.
 - $f(u_p, v_b) = 1 \Rightarrow$ we set $g(p) = b$.
 - We only added (u_p, v_b) edges when $|x_p - x_b| \leq r_p \Rightarrow$ guarantees $|x_p - x_b| \leq r_p$.
 - so the assignment is valid.

d. **Respecting station capacities.**

Now consider a station-node v_b . $f_{\text{in}}(v_b)$ is exactly the number of computers p with $g(p) = b$. (Each computer p with $g(p) = b$ has $(u_p, v_b) = 1$)

- Flow conservation $\Rightarrow f_{\text{in}}(v_b) = f_{\text{out}}(v_b)$
- Capacity constraint $\Rightarrow f_{\text{out}}(v_b) \leq c(v_b, t) = L_b$

3. **Hence the max load assigned by g to any station is at most L_b .**

3. **Conclusion**

- We have produced a function $g: P \rightarrow B$ such that
 - Every computer p goes to some $g(p)$ within its radius
 - No station b receives more than L_b computers
- That is exactly a feasible assignment.

Time

1. **Flow network construction:** Let $|B|=m$ and $|P|=n$. Building an adjacency-list G takes $\Theta(mn)$ time since: - Adding source & sink: $O(1)$; - For each peP , add u_p and edge (s, u_p) : $\Theta(n)$; - For each beB , add v_b and edge (v_b, t) of capacity L_b : $\Theta(m)$; - For each $(p, b) \in P \times B$, add (u_p, v_b) : $\Theta(mn)$

Vertices: $|V| = |B| + 2 = n + m + 2 = \Theta(m+n)$; Edges: $|E| = |B| \times |P| + n + m = \Theta(mn)$; $\Theta(mn)$ dominates.

2. **Max-flow algorithm:**

Since max flow $\leq c(s, V(s)) = |P| = n$, the FF using any reasonable augmenting path selection strategy would solve the max flow instance within $O(|E| \cdot C) = O((m+n) \cdot n) = O(mn^2)$

NP completeness (Karp reduction from NPC)

Answer Template:

General Idea

1. **Components Identification**

Instance of your problem

- Input: Objects: Computers and base stations
- Output: YES if...; Otherwise NO
- Constraints

1. A computer can only be assigned to a single base station

2. A computer can only be assigned to a base station if it is within its radius

2. **Properties Translation using components (reduction/network construction)**

Construct a flow network $G=(V, E)$ with capacity function c as follows:

- Add a source vertex s and a sink vertex t to V .
- For each computer peP , add a vertex u_p to V and an edge (s, u_p) to E , and set $c(s, u_p) = 1$.
- For each base station beB , add a vertex v_b to V and an edge (v_b, t) to E , and set $c(v_b, t) = 1$.
- For each computer-station pair $(p, b) \in P \times B$, if $|x_p - x_b| \leq r_p$, then add an edge (u_p, v_b) to E , and set $c(u_p, v_b) = \infty$.

Finally, set the target flow value to $K = |P|$. If its value equals, then all computers can be assigned (output YES); otherwise output NO.

3. **State which algorithm you use to compute the max flow.**

Use Ford-Fulkerson to compute the maximum flow K in G .

- If the resulting max-flow K is not equal to $|P|$, then no feasible assignment exists.

4. **Solution Translation**

Feasibility: If max-flow = $|P|$, every u_p sends exactly 1 unit to some v_b .

Reconstruction: For each computer p , find the station b such that the edge $u_p \rightarrow v_b$ carries 1 unit of flow. Assign p to b .

Proof

$\Rightarrow$ If there exists a feasible assignment in our original problem instance then there must exist a max flow of value n in our reduced max flow instance (e.g. a base station that is within range for every computer).

1. **Assume assignment and define a flow instance.**

- Assume there is a valid assignment $g: P \rightarrow B$ with each station's load $L_b = \begin{cases} 1, & g(p) = b, \\ 0, & \text{otherwise.} \end{cases}$
- Define a flow f on the max flow instance as follows:
 - For each computer $p \in P$, set $c(s, u_p) = 1$.
 - For each computer-station pair $(p, b) \in P \times B$ with an edge (u_p, v_b) , set $f(u_p, v_b) = 1$.
 - For each base station $b \in B$, set $f(v_b, t) = L_b$.
- By inspection of the reduction, the above steps assign a flow value to each edge in G .
- So f saturates the source cut, so $v(f) = c(s, V(s)) = |P| = K$. $v(f) = \sum_{p \in P} f(s, u_p) = |P| = K$.

2. **Check feasibility (a. capacity; b. integrality; c. conservation)**

To establish that f is a valid flow, we need to check the flow constraints.

- $f(e) \leq c(e)$ for every edge $e \in E$:
 - (s, u_p) are all assigned flow equal to their capacity.
 - (u_p, v_b) all have capacity ∞ .
 - (v_b, t) : capacity L_b is constituted of 1 from (u_p, v_b) s.t. $g(p)=b$: The amount of p , by assumption, can't be greater than L_b .
- $f_{\text{in}}(u_p) = f_{\text{out}}(u_p)$ for every $u_p \in V - \{s, t\}$:
 - At u_p : $f_{\text{in}}(u_p) = 1$; $(u_p, v_{g(p)})$ is guaranteed to exist in G ; $f(u_p, v_b) = 1 \Rightarrow f_{\text{out}}(u_p) = 1$
 - At v_b : in-flow from computer equals out-flow to t by definition.
- Integrality (and non-negativity): all $f(e)$ are integers ≥ 0 .

3. **Conclusion**

- Hence all flow-constraints are met, so f is a valid integral flow of value K .

$\Leftarrow$ If there exists a max flow of value n in our reduced max flow instance then there must exist a feasible assignment in our original problem (e.g. it must satisfy all edge capacity and flow conservation requirements)

1. **Assume the flow (Max-flow existence).**

- Assume the flow network admits an integral flow f of value $K = |P|$.
- By construction, K equals the capacity of the cut $c(s), V(s)$, so the max flow is $|P|$.

2. **Decoding to a solution**

- Flow integrality.** K is the capacity of the source cut, and by flow integrality theorem, we can assume that all flow values assigned to edges by f are integers.
- One unit per computer.** Look at a computer-node u_p . The only incoming edge to u_p is (s, u_p) of capacity 1, and the only outgoing edges are (u_p, v_b) .
 - Since the cut $c(s), V(s)$ is saturated, every (s, u_p) must carry exactly 1 unit.
 - From the flow conservation at u_p , e can deduce that $f_{\text{out}}(u_p) = f_{\text{in}}(u_p) = 1$ for each peP .
- c. Define the computer-station assignment $g: P \rightarrow B$**
 - $f_{\text{out}}(u_p) = 1 \Rightarrow u_p$ sends 1 unit in total
 - Flow integrality $\Rightarrow$ each outgoing (u_p, v_b) has integral flow.
 - $\Rightarrow$ exactly one edge (u_p, v_b) has a flow value of 1.
 - $f(u_p, v_b) = 1 \Rightarrow$ we set $g(p) = b$.
 - We only added (u_p, v_b) edges when $|x_p - x_b| \leq r_p \Rightarrow$ guarantees $|x_p - x_b| \leq r_p$.
 - so the assignment is valid.

d. **Respecting station capacities.**

Now consider a station-node v_b . $f_{\text{in}}(v_b)$ is exactly the number of computers p with $g(p) = b$. (Each computer p with $g(p) = b$ has $(u_p, v_b) = 1$)

- Flow conservation $\Rightarrow f_{\text{in}}(v_b) = f_{\text{out}}(v_b)$
- Capacity constraint $\Rightarrow f_{\text{out}}(v_b) \leq c(v_b, t) = L_b$

3. **Hence the max load assigned by g to any station is at most L_b .**

3. **Conclusion**

- We have produced a function $g: P \rightarrow B$ such that
 - Every computer p goes to some $g(p)$ within its radius
 - No station b receives more than L_b computers
- That is exactly a feasible assignment.

Time

1. **Flow network construction:** Let $|B|=m$ and $|P|=n$. Building an adjacency-list G takes $\Theta(mn)$ time since: - Adding source & sink: $O(1)$; - For each peP , add u_p and edge (s, u_p) : $\Theta(n)$; - For each beB , add v_b and edge (v_b, t) of capacity L_b : $\Theta(m)$; - For each $(p, b) \in P \times B$, add (u_p, v_b) : $\Theta(mn)$

Vertices: $|V| = |B| + 2 = n + m + 2 = \Theta(m+n)$; Edges: $|E| = |B| \times |P| + n + m = \Theta(mn)$; $\Theta(mn)$ dominates.

2. **Max-flow algorithm:**

Since max flow $\leq c(s, V(s)) = |P| = n$, the FF using any reasonable augmenting path selection strategy would solve the max flow instance within $O(|E| \cdot C) = O((m+n) \cdot n) = O(mn^2)$

NP completeness (Karp reduction from NPC)

Answer Template:

General Idea

1. **Components Identification**

Instance of your problem

- Input: the decision-problem instance
- Properties of YES instance: none
- Exist (Certificate) s.t. - (Certifier)

2. **Properties Translation using components**

Given an instance of SUBSET-SUM $V=\{v_1, v_2, \dots, v_n\}$ with target value t , let the sum of V be a . Consider the case that $t \leq a/2$. We construct a PARTITION-SUM instance $S=\{v_1, v_2, \dots, v_n, a-2t\}$.

Correctness:

- YES for subset sum $\Rightarrow$ YES for partition:** Assume V is a YES SUBSET-SUM instance, then there must exist some subset V' that sum to exactly t , where the sum of the remaining integers is $a-t$. Thus, there exists a partition of S into sets $S_1=V' \cup (a-2t)$ and $S_2=V \setminus V'$. Both of these sets sum to $a-t$.
- YES for partition $\Rightarrow$ YES for subset sum:** Assume V is a YES PARTITION-SUM instance, then there must exist a partition of S into sets S_1 and S_2 whose sum are equal. Since the sum of S is $2a-2t$, each of S_1 and S_2 sums to $a-t$. Assume S_1 is the set that contains $a-2t$. Therefore, the remaining integers in S_1 belong to V and sum to $a-(a-2t)=2t$.

To handle the case when $t > a/2$, note that there exists a subset that sums to t iff there exists a subset $a-t$. Thus in this case, the Partition instance we create consists of the integers $S=\{v_1, v_2, \dots, v_n, a-2(a-t)\}$. The rest of the argument is similar to the above.

Therefore, we have shown a Karp reduction from Positive Subset-Sum to Partition such that a Subset-Sum instance has a "yes instance" iff its corresponding Partition instance is also a "yes instance".

Time: Calculate a takes $O(n)$. Check if $t > a/2$ takes $O(1)$. Copy the integers V and add a new integer takes $O(n) + O(1)$. Therefore, the reduction is bounded by polynomial time. Since A is NP-hard, we conclude that B is also NP-hard.

Concepts:

Difference between a Cook reduction and a Karp reduction:

- Karp (many-one) computes a single instance
- Cook (Turing) makes multiple queries to an oracle for B.
- Every Karp is a special case of a Cook with just one nonadaptive query.

Simple equivalence: $\text{INDEPENDENT-SET} \leq_p \text{VERTEX-COVER}$.

Three types of standard reductions:

- Special case to general case: $\text{VERTEX-COVER} \leq_p \text{SET-COVER}$
- Encoding with gadgets: $3\text{-SAT} \leq_p \text{INDEPENDENT-SET}$
- 3-SAT $\leq_p$ CLIQUE

Decision $\leq_p$ Search $\leq_p$ Optimization

- D: Does there exist a matching of size $\geq k$? Yes/No.
- Binary Search $1 \rightarrow m$ ($O(\log m)$ calls to D)
- O: size of optimum k s.t. YES above? Number.
- Remove node by node ($O(m)$ calls to O)
- S: Find a matching of size $\geq k$. Set of edges.
- O(1) call to S

Transitivity: If $X \leq_p Y$ and $Y \leq_p Z$, then $X \leq_p Z$.

Equivalence: If $X \leq_p Y$ and $Y \leq_p X$, then $X \equiv_p Y$.

Classes (P, NP, NP-hard, NPC)

P: Decision problems for which there is a poly-time algorithm.

NP: Decision problems X which have a poly-time certifier V s.t. For every YES-instance x , there exists a certificate c such that $V(x, c) = \text{YES}$.

EXP: exponential-time; NP $\leq$ EXP.

NP-hard: A decision problem such that every problem in NP reduces to it.

NP-complete: A problem in NP such that every problem in NP polynomially reduces to it.

Induction for DP

Base Case For amount 0 we wrote down 0 coins. That is obviously optimal, because you cannot use a positive number of coins and still sum to zero cents.

Inductive Hypothesis Assume that for every amount strictly smaller than some number K (that is, for $0 \leq k < K$) the table entry $\text{dp}[k]$ equals the true minimum number of coins.

Inductive Step Consider amount K . Any legal way of making K must end with one last coin of value 1, 2, or 5. Removing that coin leaves $K-1$, $K-2$, or $K-5$ cents respectively. By the inductive hypothesis, $\text{dp}[K-1]$, $\text{dp}[K-2]$, and $\text{dp}[K-5]$ (when those indices are non-negative) already store optimal answers for those smaller amounts. Our algorithm tries each of those possibilities and picks the smallest resulting total. Call the answer it chooses A . Suppose, for contradiction, that A is not optimal. Then there exists some other arrangement that uses fewer coins. Remove its last coin; the remainder is one of the smaller amounts mentioned above, and the number of coins it used for that remainder must be strictly less than the value we stored for that remainder in dp . That violates the inductive hypothesis. Therefore the contradiction shows A must in fact be optimal.

By induction, $\text{dp}[k]$ is optimal for every K from 0 up to n , so the value the algorithm returns, $\text{dp}[n]$, is the true minimum number of coins.

Running Time The loop touches each amount once and performs a constant amount of work each time, so the total running time is proportional to n ; we say it runs in $O(n)$ time.

3. **Translating the solution from the max flow instance to the original problem instance:**

- We need to iterate through all the edges in the flow network and for each edge between a computer and base station that has a flow of 1, then output the assignment.
- $O(mn)$ edges $\Rightarrow$ this step can be done in $O(mn)$ time.

4. **Overall bound:** $O(p(n) + F(V, E))$, which is polynomial in n .

$O(mn) + O(mn^2) + O(nm) = O(mn^2)$

Ford-Fulkerson Algorithm

- Initialize f to $f(e) = 0$ for every edge e
- Compute residual network G_f
- While there exists an s - t path P in G_f
 - $f = \text{Augment}(P, f)$
 - Update G_f
- Return f

Iterations.

Path in G_f can be found in $O(m+n)$ time using BFS.

Augment(P) takes $O(n)$ time.

Updating G_f takes $O(m+n)$ time.

Application List (definition, input, certificate, certifier- ϵ , Proper):

Constraint Satisfaction problem.

SAT: 3-SAT: Given CNF formula ϕ , where each clause has exactly 3 literals, does it have a satisfying truth assignment? e.g. $\phi = (x \vee y \vee z) \wedge (\neg x \vee y \vee z)$. A propositional formula ϕ over variables $X = \{x_1, \dots, x_n\}$. A truth assignment $A: X \rightarrow \{T, F\}$. e.g. $x=F, y=T, z=F$. Check $A(x_i) \in \{T, F\}$. Check each clause in ϕ has at least one true literal.

Circuit-SAT: Given a combinational circuit built out of AND, OR, and NOT gates, is there a way to set the circuit inputs so that the output is 1?

Packing Problem:

Independent Set: Given a graph $G=(V, E)$ and integer k , is there a subset of vertices $S \subseteq V$ such that $|S| \geq k$, and for each edge at most one of its endpoints is in S ? e.g. 1-2-3-4-1, $k=2$. $G=(V, E)$, integer k . A subset $S \subseteq V$ with $|S| \geq k$. Check $|S| \geq k$. Check $vu, vu \in S$: (u, v) not in E .

Clique: (A clique is a complete subgraph) Given a graph $G=(V, E)$ and integer k , does G contain a clique of size k ? $G=(V, E)$, integer k . A subset $C \subseteq V$ with $|C| \geq k$. Check $vu, vu \in C$ and $uv \in E$ for all $(u, v) \in C$.

Steiner Tree: Given a graph $G=(V, E)$ and subset of vertices X in that graph, an integer k , does there exist a subset $S \subseteq V$ of size at most k such that $G[X \cup S]$ is connected? e.g. $V=\{1, 2, 3, 4\}$, $E=\{(1, 4), (2, 4), (3, 4)\}$, $X=\{1, 2, 3\}$, $k=4$. $S \subseteq V$ with $|S| \leq k$.

Feedback Set: Given a directed graph $G=(V, E)$ a feedback set is a set $X \subseteq V$ with the property that $G-X$ is acyclic. Given G and k , does G have a feedback set of size at most k ? $G=(V, E)$, integer k . A subset $S \subseteq V$ with $|S| \leq k$. Remove S and check the remaining graph has no cycle.

Covering Problem:

Set-Cover: Given a set U of elements, a collection $S_1, S_2, \dots, S_m$ of subsets of U , and an integer k , does there exist a collection of k of these sets whose union is equal to U ? $U=\{S_1, \dots, S_m\}$, integer k . $C \subseteq \{1, \dots, m\}$ with $|C| \leq k$. Check $\bigcup_{i=1}^k S_{C_i} = U$.

Vertex Cover: Given a graph $G=(V, E)$ and integer k , is there a set of at most k vertices $\{v_1, \dots, v_k\}$ such that every edge $e \in E$ has at least one endpoint in $\{v_1, \dots, v_k\}$. Check the k vertices $\{v_1, \dots, v_k\}$ such that every edge $e \in E$ has at least one endpoint in $\{v_1, \dots, v_k\}$. Check the k vertices $\{v_1, \dots, v_k\}$ such that every edge $e \in E$ has at least one endpoint in $\{v_1, \dots, v_k\}$. Check the k vertices $\{v_1, \dots, v_k\}$ such that every edge $e \in E$ has at least one endpoint in $\{v_1, \dots$